BanX: A Hybrid Crypto-Backed and Crypto-Collateralized Stablecoin Protocol – Protocol Description – v0.9

Zahnentferner
The Stable Order
zahnentferner@gmail.com

November 4, 2024

Abstract

BanX[1] is a hybrid crypto-backed and crypto-collateralized stablecoin protocol. Because **BanX** is a crypto-collateralized stablecoin protocol, users may mint and borrow the stablecoin by providing a collateral asset. If the loan's collateral ratio drops below a threshold, the loan is liquidated and the collateral is sold at a discount to users who bid the stablecoin for the collateral. And, because **BanX** is also a crypto-backed stablecoin protocol, stablecoin holders can redeem their stablecoins at any moment against the collateral of the worst loan. Therefore, the collateral can also be considered a reserve for the stablecoin.

1 Introduction

Crypto-collateralized stablecoin protocols (e.g. [1]), where users borrow freshly minted stablecoins in the form of an over-collateralized loan providing a crypto asset as collateral, are among the oldest and still most popular types of stablecoins nowadays. Their popularity is likely due to their battle-tested stability and the fact that they directly address one of the most common use cases for stablecoins: borrowing and then using what you borrowed to buy more of the collateral asset, thereby leveraging your position in relation to that collateral asset.

[1] This research was funded by AlphBanX.

Their stability arises from the following mechanism: if the price of the collateral asset falls and the collateralization ratio of a loan drops below a well-defined threshold, the loan can be liquidated and collateral from that loan is used to buy and burn stablecoins.

And, yet, this mechanism also reveals an obvious flaw of stablecoin protocols of this type. Since this mechanism only works when loans are getting liquidated, there is nothing in the protocol buying stablecoins for its target nominal price in other circumstances, when no loans are being liquidated. Consequently, a user wishing to sell the stablecoin must resort to doing it in a secondary market, and not directly with the protocol, and the prices in secondary markets are in no way guaranteed to be stable. There might not even be anyone willing to buy the stablecoins in the secondary markets.

Although this flaw is not openly discussed, it may explain why there is a trend (e.g. [2]) to create stablecoin protocols that are hybrid between crypto-collateralized and crypto-backed. Although these two types are often confused, the latter (e.g. [3–5]) has a reserve (and not collateral) and users can redeem their stablecoins for a portion of the reserve at any time.

In this paper, we describe the **BanX** protocol, a hybrid crypto-collateralized and crypto-backed stablecoin protocol where users can always redeem their stablecoins against the collateral of the worst loan, where loans are ranked by the interest rate that they are paying.

2 Protocol Description

2.1 Notations

- CA = Collateral asset
- SC = Stablecoin
- BT = Bank Token
- For any variable or mutable object v, we denote by $v[\tau]$ that variable or mutable object at time τ.
- For any address a and any asset x, we denote by $B(a, x)$ the balance of x in a.
- For any user or contract u, we denote by $addr(u)$ its address.
- for any token T, we denote by $supply(T)$ its total supply.

2.2 Definitions

The *oracle price* P is the price of the peg asset (e.g. USD) in relation to the underlying collateral asset.

Every *loan* ℓ is a contract with the following variables:

- the loan's owner's address, denoted $owner(\ell)$
- the quantity of stablecoins borrowed, denoted $q(\ell)$
- the interest rate, denoted $ir(\ell)$
- the timestamp of the last interest payment, denoted $t(\ell)$

A loan's *collateral* $c(\ell)$ is defined as:

$$c(\ell) \doteq B(addr(\ell), CA)$$

A loan's *health ratio* $h(\ell)$ is defined as:

$$h(\ell) \doteq \frac{c(\ell)}{q(\ell)}$$

A loan's *collateral ratio* $r(\ell)$ is defined as:

$$r(\ell) \doteq \frac{c(\ell)}{q(\ell)P}$$

A loan's *owed interest* $interest(\ell)[\tau]$ is defined as:

$$interest(\ell)[\tau] \doteq q(\ell)\ ir(\ell)\ P[\tau]\ (\tau - t(\ell))$$

Every *auction bid* b has the following variables:

- the bid's owner's address, denoted $owner(b)$
- the amount of stablecoins offered for sale, denoted $q(b)$
- the requested discount, denoted $d(b)$

The bank B is a contract with a list $\mathcal{L}$ of loans ordered by their interest rates and a list $\mathcal{B}$ of auction bids ordered by their requested discounts.

2.3 Parameters

- Minimum Collateral Ratio: ρ_{min}
- Minting Fee: φ_m
- Redemption Fee: φ_r
- Liquidation Fee: φ_l
- Successful Bid Fee: φ_b
- Bid Closure Fee: φ_{bc}
- Liquidator Reward: μ

2.4 Protocol Operations

The following subsections define all the operations that can be executed through the bank. In all these operations, we denote by τ the time just before the operation and by τ' the time just after the operation.

2.4.1 Create Loan

- Arguments for the operation:
  - amount of stablecoins to be borrowed n.
  - amount of provided collateral m.
  - desired interest rate i.
  - Who can execute this operation: any user or contract u.
- Effects of the operation:
  - Transfer m CAs from $addr(u)$ to $addr(\ell)$.
  - Mint n SCs to $addr(u)$.
  - Transfer $n\varphi_m P[\tau]$ CAs from $addr(\ell)$ to $addr(B)$.
  - Create a loan $\ell \doteq (addr(u), n, m, i, \tau)$.
  - Insert ℓ in $\mathcal{L}$.
- Requirements of the operation:
  - $r(\ell) \geq \rho_{min}$

2.4.2 Close Loan

- Arguments for the operation:
  - loan ℓ to close.
- Who can execute this operation: any user or contract u.
- Effects of the operation:
  - Transfer $c(\ell)$ CAs from $addr(\ell)$ to $owner(\ell)$.
  - Remove ℓ from $\mathcal{L}$.
- Requirements of the operation:
  - $q(\ell) = 0$

2.4.3 Borrow More

- Arguments for the operation:
  - loan ℓ from which to borrow more.
  - amount of stablecoins to be borrowed n.
- Who can execute this operation: only the loan's owner $owner(\ell)$.
- Effects of the operation:
  - Mint n SCs to $owner(\ell)$.
  - Transfer $n\varphi_m P[\tau]$ CAs from $addr(\ell)$ to $addr(B)$.
  - $q(\ell) \Leftarrow q(\ell) + n$
- Requirements of the operation:
  - $r(\ell[\tau']) \geq \rho_{min}$

2.4.4 Repay

- Arguments for the operation:
  - loan ℓ to repay.
  - amount of stablecoins to repay n.
- Who can execute this operation: only the loan's owner $owner(\ell)$.
- Effects of the operation:
  - Burn n SCs from $owner(\ell)$.
  - $q(\ell) \Leftarrow q(\ell) - n$
  - If $q(\ell) = 0$, Close ℓ. Else reposition ℓ in $\mathcal{L}$.
- Requirements of the operation:
  - $n \leq q(\ell[\tau])$

2.4.5 Withdraw Collateral

- Arguments for the operation:
  - loan ℓ from which to withdraw collateral.
  - amount of collateral to withdraw m.
- Who can execute this operation: only the loan's owner $owner(\ell)$.
- Effects of the operation:
  - Transfer m CAs from $addr(\ell)$ to $owner(\ell)$.
- Requirements of the operation:
  - $r(\ell[\tau']) \geq \rho_{min}$

2.4.6 Add Collateral

- Arguments for the operation:
  - loan ℓ to which to add collateral.
  - amount of collateral to add m.
- Who can execute this operation: only the loan's owner $owner(\ell)$.
- Effects of the operation:
  - Transfer m CAs from $owner(\ell)$ to $addr(\ell)$.
- Requirements of the operation:
  - None

2.4.7 Charge Interest

- Arguments for the operation:
  - loan ℓ from which to charge interest.
- Who can execute this operation: any user or contract u.
- Effects of the operation:
  - Transfer $interest(\ell)[\tau]$ CAs from $addr(\ell)$ to $addr(B)$
- Requirements of the operation:
  - None

2.4.8 Redeem

- Arguments for the operation:
  - amount n of SCs to redeem.
- Who can execute this operation: any user or contract u.
- Effects of the operation:
  - Burn n SCs from $addr(u)$
  - Initialize $k = n$ and, while $k > 0$, do:
    - Let ℓ be the first loan in $\mathcal{L}$.
    - Let $v \doteq k$, if $k \leq q(\ell)$, or $v \doteq q(\ell)$, otherwise.
    - Let $w \doteq vP'[\tau](1 - \varphi_r)$, where:
      - P' is the redemption price and it is defined as:

$$P' \doteq P \; min\left(1, \frac{\sum_{\ell \in \mathcal{L}} B(addr(\ell), CA)}{P \sum_{\ell \in \mathcal{L}} q(\ell)}\right)$$

    * Transfer w CAs from $addr(l)$ to $addr(u)$.
    * $q(\ell) \Leftarrow q(\ell) - v$
    * If $q(\ell)[\tau'] = 0$, Close ℓ.
    * $k \Leftarrow k - v$

- Requirements of the operation:
  - $B(addr(u), SC) \geq n$

2.4.9 Create Bid

- Arguments for the operation:
  - amount of stablecoins offered for sale n.
  - the requested discount d.
  - Who can execute this operation: any user or contract u.
- Effects of the operation:
  - Create a bid $b \doteq (addr(u), n, d)$.
  - Transfer n SCs from $addr(u)$ to $addr(b)$.
  - Insert b in $\mathcal{B}$.
- Requirements of the operation:
  - None.

2.4.10 Close Bid

- Arguments for the operation:
  - bid b to close.
- Who can execute this operation: any user or contract u.
- Effects of the operation:
  - Transfer $(1 - \varphi_{bc})q(b)$ SCs from $addr(b)$ to $owner(b)$.
  - Burn $\varphi_{bc}q(b)$ SCs from $addr(b)$.
  - Remove b from $\mathcal{B}$.
- Requirements of the operation:
  - None.

2.4.11 Liquidate Loan

- Arguments for the operation:
  - loan ℓ to liquidate.
- Who can execute this operation: any user or contract u.
- Effects of the operation:
  - While $\mathcal{B}$ is non-empty and $r(\ell) < \rho_{min}$:
    - Let b be the first bid in $\mathcal{B}$.
    - If $q(b) > q(\ell)$:
      - Let $n \Leftarrow min(c(\ell), q(\ell)P(\tau)(1 + d(b)))$
      - Transfer $\mu(1 - \varphi_b)n$ CAs from $addr(\ell)$ to $owner(u)$
      - Transfer $(1 - \mu)(1 - \varphi_b)n$ CAs from $addr(\ell)$ to $owner(b)$
      - Transfer $\varphi_b\ n$ CAs from $addr(\ell)$ to $addr(B)$
      - Transfer $min(c(\ell), \varphi_l q(\ell)P(\tau))$ CAs from $addr(\ell)$ to $addr(B)$
      - $q(b) \Leftarrow q(b) - q(\ell)$
      - $q(\ell) \Leftarrow 0$
      - Close ℓ
    - Otherwise:
      - Transfer $\mu min(c(\ell), q(b)P(\tau)(1+d(b)))$ CAs from $addr(\ell)$ to $owner(u)$
      - Transfer $(1-\mu)min(c(\ell), q(b)P(\tau)(1+d(b)))$ CAs from $addr(\ell)$ to $owner(b)$
      - $q(\ell) \Leftarrow q(\ell) - q(b)$
      - $q(b) \Leftarrow 0$
      - Close b
- Requirements of the operation:
  - $r(\ell[\tau]) < \rho_{min}$

Note: as defined above, bidders may receive less than they desire if there is not enough collateral in the liquidated loan. For example, if a loan's collateral ratio drops to 110% and the bidder wants a discount of 20%, the bidder will only get a discount of 10%.

2.4.12 Distribute Rewards

- Arguments for the operation:
  - None.
- Who can execute this operation: any user or contract u.

- Effects of the operation:
  - Let k be the ratio of SCs in bids by the total supply of SCs:

$$k \doteq \frac{\sum_{b \in \mathcal{B}} q(b)}{supply(SC)}$$

  - Transfer $(1-k)\ B(B, CA)$ CAs from $addr(B)$ to SC bidders, proportionally to how much they are bidding.
  - Transfer $k\ B(B, CA)$ CAs from $addr(B)$ to BT stakers, proportionally to how much they are staking.
- Requirements of the operation:
  - None.

3 Discussion

The stability of stablecoins in the **BanX** protocol is guaranteed by the redemption mechanism: Users are always able to redeem their stablecoins for a portion of the collateral/reserve. The protocol is always willing to buy back the stablecoins from users. The price at which the protocol buys back the stablecoins is the peg price minus a redemption fee, as long as the total collateral ratio is above 100%.

Most operations charge fees. All fees aim to provide rewards for stakers and bidders, to generate demand for stablecoins and bank tokens. Besides the reward aspect, each fee also serves other purposes. The redemption fee protects against oracle delays and manipulation, and acts as a reward to borrowers whose collaterals are serving as a reserve for redemptions. The liquidation fee penalizes borrowers who let their loans get liquidated. The bid closure fee discourages bidders from closing their bids and dumping stablecoins in the market. The bid success fee ensures that all bidders earn a bit, even while their bids are not successful. This keeps them bidding.

The dynamic distribution of rewards among BT stakers and SC bidders creates a negative feedback mechanism that should prevent a bubble of demand for SCs. As the number of SCs used in bids grows in relation to the total supply of SCs, the fraction of rewards sent to bidders is reduced, discouraging further bidding.

Loans with low interest rate are prioritized for redemptions because they bring less value to the system.

4 Disclaimer

This is a protocol description paper. All arguments presented here are for information and research purposes only. You assume all risks of using implementations and deployments of this protocol. The author will not be liable for

any indirect, special, incidental or consequential damages of using any software based on or derived from information contained in this paper.

References

[1] "The Maker Protocol: MakerDAO's Multi-Collateral Dai (MCD) System," [Online], 2017, https://makerdao.com/en/whitepaper/.

[2] Liquity, "Liquity v2," 2024, https://liquity.gitbook.io/v2-whitepaper.

[3] J. Zahnentferner, D. Kaidalov, J.-F. Etienne, and J. Díaz, "Djed: A formally verified crypto-backed pegged algorithmic stablecoin," Cryptology ePrint Archive, Report 2021/1069, 2021, https://eprint.iacr.org/2021/1069.

[4] ——, "Djed: A formally verified crypto-backed pegged autonomous stablecoin," in *IEEE International Conference on Blockchain and Cryptocurrency*, 2023, https://icbc2023.ieee-icbc.org/program/icbc-2023-final-program.

[5] J. Zahnentferner, L. D'angelo, and M. Shaheer, "Gluon: A cryptocurency stabilization protocol," 2024, .

This work is licensed under the CreativeCommons Attribution-NonCommercial-NoDerivatives 4.0 International license[2].

For derivative and commercial uses, please contact the authors or AlphBanX.

[2] https://creativecommons.org/licenses/by-nc-nd/4.0/